

SQL 注入漏洞修复报告

用户管理系统 · Day3 作业

班 级：_____

姓 名：汪乙函

学 号：_____

指导教师：_____

提交日期：2026年07月08日

一、概述

1.1 作业内容

在 Day2 密码安全修复的基础上，针对用户管理系统中新增的注册功能和搜索功能进行SQL注入漏洞审计与修复。本次作业重点排查了系统内所有与 SQLite 数据库交互的代码路径，发现并修复了 2 处 SQL 注入漏洞和 1 处数据库错误信息泄露问题。

1.2 测试环境

项目	说明
目标系统	用户管理系统 (Flask + SQLite)
测试地址	https://192.168.184.131:5000
数据库	SQLite 3 (data/users.db)
测试方法	黑盒注入测试 + 白盒代码审计
GitHub	github.com/894728749/user-management-system

二、漏洞发现总览

编号	漏洞名称	风险等级	修复状态
SQLI-001	搜索功能 UNION/OR 注入	高危	已修复
SQLI-002	注册功能 INSERT 注入	高危	已修复
SQLI-003	数据库错误信息泄露	低危	已修复

三、漏洞详情与修复过程

SQLI-001：搜索功能 UNION/OR 注入

【漏洞描述】

搜索功能接收用户输入的 keyword 参数后，直接通过 f-string 拼接进 SQL 查询语句。攻击者可以在搜索框中输入单引号、UNION 关键字或 OR 条件来篡改原始 SQL 语句的执行逻辑。

【漏洞位置】

app.py -> search() 路由

【未修复前的危害】

危害一：UNION 注入获取任意数据

攻击者输入：

```
' UNION SELECT 1,'inj','inj@x.com','138'--
```

UNION 将第二个查询结果合并到原始结果集，页面出现 "inj" 用户名。

危害二：OR 注入搜索全部用户

攻击者输入：

```
' OR '1'='1
```

永真条件导致返回 users 表全部记录，泄露所有用户名、邮箱、手机号。

【修复思路】

将 f-string 拼接替换为参数化查询，? 占位符避免用户输入被当作 SQL 指令解析。

【修复代码】

修复前：

```
sql = f"SELECT ... WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
conn.execute(sql)
```

修复后：

```
sql = "SELECT ... WHERE username LIKE ? OR email LIKE ?"
conn.execute(sql, (like_pattern, like_pattern))
```

【修复后验证】

测试用例	修复前	修复后
' UNION SELECT 1,'inj','inj@x.com','138'--	出现 "inj" 用户名	返回"无搜索结果"
' OR '1'='1	显示全部用户	返回"无搜索结果"
正常搜索 admin	显示 admin	显示 admin

SQLI-002：注册功能 INSERT 注入

【漏洞描述】

注册功能同样使用 f-string 拼接。攻击者可在用户名中注入额外 SQL 语句。

【漏洞位置】

app.py -> register() 路由

【未修复前的危害】

攻击者输入用户名：

```
hacker', 'pass', 'h@x.com', '123')--
```

单引号闭合 VALUES 左括号，-- 注释后续语句，插入攻击者可控的数据。

【修复思路】

使用参数化查询替换 f-string 拼接。

【修复代码】

修复前：

```
sql = f"INSERT INTO users VALUES ('{username}', '{password}', '{email}', '{phone}')"
conn.execute(sql)
```

修复后：

```
sql = "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)"
conn.execute(sql, (username, password, email, phone))
```

【修复后验证】

测试用例	修复前	修复后
hacker', 'pass', 'h@x.com', '123')--	插入恶意数据	提示"注册失败"
正常注册新用户	插入正常用户	插入正常用户

SQLI-003：数据库错误信息泄露

【漏洞描述】

注册功能错误处理直接将 SQLite 原始错误返回给用户，暴露表名和字段名。

【漏洞位置】

app.py 第 416 行：error=f"注册失败：{e}"

【未修复前的危害】

攻击者可通过注册已知用户名触发 UNIQUE 错误，完成用户名枚举。

【修复代码】

修复前：

```
return render_template("register.html", error=f"注册失败：{e}")
```

修复后：

```
return render_template("register.html", error="注册失败，用户名可能已存在")
```

四、SQL 操作安全审计总表

对项目中的所有 SQLite 操作逐条审查：

数据库文件	SQL 操作	方式	状态
logs/rate_limit.db	SELECT 锁定检查	占位符？	安全
logs/rate_limit.db	SELECT 速率计数	占位符？	安全
logs/rate_limit.db	INSERT 失败记录	占位符？	安全
logs/rate_limit.db	INSERT 账号锁定	占位符？	安全
logs/rate_limit.db	DELETE 过期清理	占位符？	安全
data/users.db	INSERT 默认用户	占位符？	安全
data/users.db	INSERT 注册用户	占位符？	已修复
data/users.db	SELECT 搜索用户	占位符？	已修复
data/users.db	错误信息展示	通用提示	已修复

五、修改文件清单

文件	操作	说明
app.py	修改	搜索/注册改为参数化查询；错误信息改为通用提示
CHANGELOG_SECURITY.md	修改	新增 SQL 注入修复记录

六、心得体会

通过本次作业，我对 SQL 注入漏洞的形成原理和防御手段有了更直观的理解。

- 1. SQL 注入产生的根本原因是用户输入被当作代码执行。参数化查询能将输入与 SQL 指令分离。
- 2. 安全编码规范必须全局执行，一处疏漏就可能导致全盘失守。
- 3. 辅助信息（如数据库错误）也可能帮助攻击者，修复要彻底。
- 4. 三个 POC 验证了修复效果，说明黑盒测试在安全审计中的重要性。

— 报告结束 —